FLOWI · AI INTELLIGENCE

— FIELD GUIDE · VOLUME 03

The AI Builder's Field Guide

Four pieces on what shipped in AI this month — and what the production patterns mean for you.

USEFLOWI.APP · 2026

— CONTENTS

The AI Builder’s Field Guide

— FLAGSHIP RELEASE

Claude Mythos found hundreds of Firefox bugs. The real story is the triage.

By **Flowi Editorial** · *May 9, 2026 · 6 min read*

Mozilla used Claude Mythos to find vulnerabilities in Firefox. The interesting shift is what happened to the security team's workflow once the bug volume changed.

A security team that used to find one critical vulnerability a week now finds twelve in a morning. That's the shift Mozilla just published — quietly, in a hacks.mozilla.org post — about what happened when they pointed Anthropic's Claude Mythos preview at the Firefox source tree.

The headline number ("hundreds of vulnerabilities found and fixed") is the wrong thing to focus on. The interesting story is what changed in **how Mozilla's security workflow runs** once the cost of finding bugs dropped to roughly zero. Because that's the part that translates to your codebase, whether or not you ever get a Mythos preview key.

What launched

Anthropic gave Mozilla preview access to Claude Mythos — the version of Claude that's been spending an unusually long time on a single research task before reporting back. Mozilla pointed it at the Firefox C++ source, the Rust modules, the JavaScript engine, and the WebAssembly runtime. They asked it to find memory-safety bugs, race conditions, and integer overflow paths.

What came back was, in Simon Willison's framing, "suddenly the bugs are very good." Mythos didn't just produce a wall of false-positive grep hits. It produced reports that included a hypothesized exploit path, a minimal reproducer, and a suggested patch. Three of those things are usually the bottleneck on a security ticket.

Mozilla published a blog post. Buried in it: the team had to invent a new triage tier because their existing severity ladder was designed for a world where vulnerabilities arrived one at a time, on a Friday afternoon, from a lone external researcher. Mythos delivered them in batches of fifteen, on a Tuesday.

Why it matters

The bottleneck in security work was never finding bugs. Static analysis tools have been finding bugs for thirty years. The bottleneck was deciding **which of the 8,000 reported issues are real and worth fixing** before the engineers stop returning your DMs.

Three things have to be true for a bug report to actually get fixed:

1. The bug is real (most aren't — false positives have always been the dominant failure mode of automated security tooling).
2. Someone can reproduce it cheaply (a one-line "potential nullptr deref in handler.cpp:412" is not actionable).
3. Someone can patch it without breaking three other things.

Mythos is closing all three at once. The reports include the exploit path, so the bug is provably real. They include a minimal reproducer, so engineering can re-run it locally and watch it fail. They include a candidate patch, so the engineering decision becomes "merge this" rather than "investigate."

When you collapse those three steps from "two days of senior security engineer time" into "fifteen minutes of code review," the math of how a security team is staffed changes. Mozilla's blog post mentions this in passing — "we've moved security review earlier in the cycle" — but the operational implications are bigger than that line suggests.

The triage shift nobody is talking about

Here's what you don't see in the Mozilla post but probably should: the security team's job has structurally changed. They're no longer the people who **find** vulnerabilities. They're the people who **rank** vulnerabilities — what fixes ship this release, what fixes can wait, what's a real attack path versus a theoretical one.

That's a different skill profile. The senior security engineer of 2024 was, broadly, an exploit researcher who could think like an attacker. The senior security engineer of 2026 needs to be a **threat-model strategist** who can read fifty Mythos reports a day and decide which three matter most for the threat model of the next release.

This isn't a hypothetical. It's what Mozilla had to actually build a process around in real time, on a preview product, in the last three months. The blog post describes a new internal tier they call "Mythos-class" — a queue of bugs that are real, are exploitable, but for which the priority is set by **product impact** rather than discovery date.

That's the shift you should be paying attention to: the security team has been promoted to threat-model owners. The bug-finding work is now a commodity input.

What this means for your codebase

You probably don't have Mythos preview access. That's fine. The lessons from Mozilla's experience apply anyway, because the trajectory is one-way: AI vulnerability scanners are about to become a baseline expectation in any serious security program. Anthropic, OpenAI, and the open-weight models are all converging on the same capability. The question is when, not if, you can run something Mythos-class against your own code.

When that day arrives, three things will hurt your team if you haven't prepared:

The triage tier doesn't exist. Most teams' bug trackers have "P0 / P1 / P2" tiers that were calibrated for a world where one P0 came in per week. When fifty come in per morning, your tier definitions need to be product-impact-aware, not severity-aware.

Reproducer environments aren't reproducible. A security report with a reproducer is only useful if the reproducer runs. Most internal tooling assumes a developer's local machine, with a particular Docker compose file, with particular env vars. The AI scanner doesn't have that context. Your reproducer infrastructure needs to be self-contained and one-command-to-run, or the velocity advantage evaporates the moment a developer has to spend forty minutes setting up their environment.

The "merge this patch" muscle is weak. Most security fixes today are written by the engineer who owns the file, often two weeks after the original report. When the AI generates a

candidate patch, your code review process needs to be set up to **evaluate the patch at the source-language level**, not "tell the engineer to investigate." That requires reviewers who can read security-class code changes critically — which most engineering teams have outsourced, increasingly, to "the security team."

Who should care

- **Security engineers shipping in production:** start writing your team's playbook for when bug volume goes 10x. The shift from finder to ranker is going to land within twelve months.
- **CTOs and engineering managers:** if your security team is staffed for "find bugs," restaff. Find-bugs is being commoditized. Rank-bugs is the work.
- **Solo and small-team builders:** AI security review is going to become table-stakes for any code you ship to production. Start running Claude Code on your repo with security-focused prompts now, so the muscle is built before the volume arrives.

The Mozilla post will be remembered as the first public deployment write-up of an AI security tool that genuinely changed how a major engineering org operates. The hundreds-of-bugs-fixed number is impressive. The reorganization of the team that fixed them is more so.

If you're building AI-powered systems in production — agents, automations, security tooling — the patterns Mozilla just landed on are the same patterns you'll need. The decision tree for how an agent surfaces work, prioritizes it, and hands it off cleanly to a human reviewer is the bottleneck across every production AI system right now. That's why we wrote Agent Memory: The 5 Patterns That Ship in Production — the failure modes aren't unique to security.

— FLAGSHIP RELEASE

OpenAI gates 'Spud' to cyber defenders. The gating IS the product.

By **Flowi Editorial** · *May 9, 2026 · 6 min read*

OpenAI is rolling out GPT-5.5 — internally codenamed Spud — to vetted security teams only. The interesting bit isn't the model's capabilities. It's the access tier.

OpenAI is releasing GPT-5.5, internally nicknamed "Spud," only to vetted cyber-defense teams. That's the framing in Axios's report Thursday. Most of the coverage will focus on the model's capabilities — it's reportedly close to Anthropic's Mythos at finding software vulnerabilities. The more interesting story is the **access architecture**, because that's the new product surface.

What launched

OpenAI is making a more permissible variant of GPT-5.5 — Spud — available, but only to companies that pass an enterprise security review and sign restricted-use contracts. The model can do offensive-security work that the public ChatGPT Plus tier won't touch: exploit chain analysis, vulnerability discovery in C/C++ codebases, fuzzer harness generation, the usual list.

This is the second major frontier release in 2026 to ship behind a tier rather than into the consumer fan-out. The first was Anthropic's Mythos, which Mozilla used to harden Firefox and which I wrote about yesterday. Mythos went to a small set of preview partners. Spud is going to a vetted-defender list. Both labs are converging on the same product shape.

Why it matters

The frontier labs have spent two years figuring out that **capability is a leading indicator, but distribution is the moat**. The reason Spud isn't on chatgpt.com is not because OpenAI worries the model will tell college kids how to write SQL injection. The reason is that "vetted access to Spud" is itself a product. Companies will pay to be on the list. Government agencies will pay to be on the list. The list is the moat.

What you're watching, in real time, is the bifurcation of the frontier model market into three tiers:

Tier 1: Public consumer. ChatGPT Plus, Claude.ai, Gemini Advanced. The model is distilled, RLHF'd into a friendly assistant, and capability-capped on the security-sensitive verbs. This is where most of the revenue is, and most of the safety energy is. It's also where the product is the most commoditized — every lab is racing to a near-identical capability ceiling.

Tier 2: Enterprise standard. Anthropic's API, OpenAI's API, Google Vertex. Higher rate limits, better availability SLAs, specific compliance certifications. The model is mostly the same as the consumer version, just with the enterprise contract attached. Distribution differentiator: integrations, billing, support.

Tier 3: Vetted-defender / vetted-research. This is the new tier. Mythos preview, Spud rollout, and presumably whatever Google's lab is testing internally. The model is **more capable** than the public tier — specifically on tasks the public tier was trained to refuse — but the access is gated by who you are, not what you pay. This is a reputation-based market.

The interesting move is that Tier 3 is not the most expensive tier. It's the **most exclusive**. Most security firms can't pay their way in. They have to demonstrate that they're a defender, not an attacker, that they have a public track record, that their disclosure policy is sane. OpenAI and Anthropic are both vetting this manually.

Show the mechanism

Why this gating is the product, in concrete terms:

A defender team on the vetted list can run Spud against their codebase and ask it to think like an attacker. The model will produce a hypothesized exploit chain, complete with reproducer and patch. That's enormous time savings — turning a senior security engineer's two-week investigation into a fifteen-minute review.

But Spud's exploit-chain-finding capability is a dual-use technology. Run it against an open-source library you don't maintain — say, a popular npm package — and you have a 0-day attack pipeline. The same skill that helps Mozilla harden Firefox helps a state-affiliated actor compromise a critical-infrastructure dependency.

OpenAI knows this. So does Anthropic. The labs' bet is that the way to ship the capability without becoming the supply chain for a wave of 0-days is to **make the access list the product**. If you're on the list, you've sworn to use Spud only on code you have permission to test. If you violate that, you lose access permanently, and your name goes into a "do not re-grant" log shared (informally) across labs.

That's a soft-power containment strategy. It's also, structurally, a return to the way frontier biotechnology was distributed in the 1980s — capability gated by reputation and signed-paperwork, not by capability cap.

What this means for builders

If you're building security tooling, three things change:

1. **The defender market just got bigger.** Mid-tier security firms that previously had to staff a senior security researcher to do offense simulation can now subscribe to a vetted-AI tier and get equivalent output for a tenth of the cost. The companies that close their access-list applications first will be the ones with brand and track record. Build that *now*, before you need it.

2. **Detection has to keep pace.** If Spud-class tools are in defenders' hands, they're also — within twelve months — in attackers' hands, either through leakage or through equivalent open-weight models like z-lab's Gemma-4 series. Defensive monitoring needs to assume the

attacker has the same offensive AI capability you do. The detection model's job is no longer "is this attempt unusual" but "is this attempt **AI-generated unusual**." Different signature.

3. **The build-vs-buy axis tilts toward buy.** If your team was considering building an internal AI-assisted code-review pipeline, that work just got economically dominated. The labs are going to ship something better in six months, gated to defenders, that you can subscribe to. Spend the engineering hours on the integration, the workflow, and the triage — not on the underlying scanner.

Who should care

- **Security engineers and SOC leads:** apply for Spud access *immediately* even if you're not sure you'll use it. The application process is part of the gating signal. Be in the queue.
- **CTOs at infrastructure-critical companies:** the vetted-defender tier is going to become a procurement line item within twelve months. Start the conversation with OpenAI and Anthropic enterprise sales now, because the queue is going to grow fast once the first big breach gets attributed to "the attackers had Spud-class tooling and the defender did not."
- **Indie builders and early-stage founders:** you're not on the vetted list and you won't be soon. That's fine. Build using the public-tier models, write security-aware prompts, and make sure your own deps are clean before someone else's vetted scanner finds something embarrassing.

OpenAI's Spud rollout is the second public datapoint that frontier-grade security capability is shipping in 2026 — and that the labs intend to control the supply chain of that capability through reputation gating. Watch for the third datapoint (Google's lab will not be far behind) and watch for the first leak (the model weights, the API keys, or the access list itself).

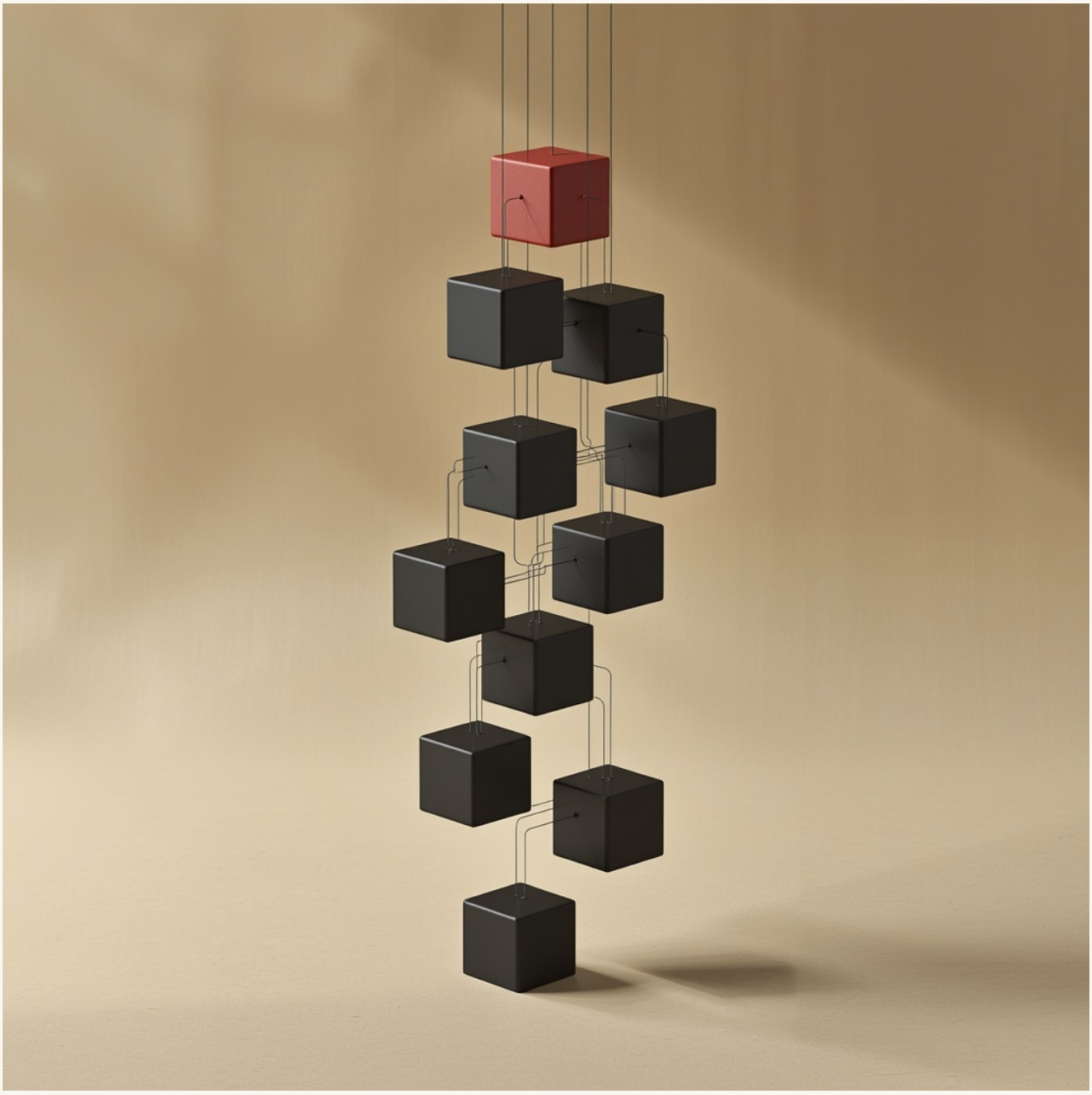
If you're building agentic systems that need to reason about code, security, or anything else where a careless tool call can compromise the user, the patterns are the same as everything we cover. The decision tree for **when** an AI agent calls a sensitive capability is the entire game right now. We dug into the production patterns in Agent Memory: The 5 Patterns That Ship in Production — the same gating logic Spud uses on the model side, you need on the tool side.

— AI CODING

Code w/ Claude 2026: what actually shipped, what was theater

By **Flowi Editorial** · *May 9, 2026 · 7 min read*

Anthropic's Code w/ Claude developer event landed yesterday. Most coverage will be hot-take. Here's the read on what actually changes for builders.



Anthropic ran their second Code w/ Claude event yesterday. There were no Project Stargate-style announcements, which the live-blog set treated as a disappointment. They're misreading the room. The interesting story is what Anthropic shipped *quietly*, between the keynote talks, that changes how you'll work with Claude Code in production over the next six months.

Here's the practitioner read — what landed, what was theater, what to update your workflow around this week.

What actually shipped

Three things, in order of how much they'll change your day:

1. Background sub-agents that can run for hours. This is the big one and Anthropic buried it. Claude Code can now spawn background agents that operate on a separate execution context, take a defined task, and report back when they're done — minutes or hours later. The classic example: "audit this entire monorepo for security antipatterns and put the results in a markdown file." That used to be a 10,000-token context-eating task that risked drowning your main session. It's now a fire-and-forget call, and the parent session continues unaware.

The implication is bigger than time savings. **You can now build agents that have agents.** A planning agent calls a research sub-agent to dig into a topic; while it's gone, the planner does other work; when the sub-agent returns, the planner integrates the result. That's the multi-agent pattern people have been hand-rolling with the Agent SDK for a year. Now it's a primitive in Claude Code itself.

2. The "skill" system, formalized. Skills were a research preview last quarter. They're now a first-class feature with a defined directory structure, a manifest format, and a repo of community-contributed skills you can install. The spec covers things like: multi-step debugging routines, library-specific code patterns, project-specific conventions. The good news is you can write a skill once and have it apply across every Claude session you start. The trap is that skills can be invoked invisibly, which makes "why did Claude do that?" harder to debug. Read the manifest of any skill you install before letting it run; this is exactly the kind of lever malicious actors will eventually try to abuse.

3. A real plan-mode hand-off. Plan mode in Claude Code now produces a *durable artifact* — a markdown plan file you can review, share with a teammate, and re-feed to Claude tomorrow. Previously the plan lived in conversation context and decayed when the session ended. The hand-off is the missing piece for actual team workflows: senior engineer scopes the plan in a 20-minute session, hands it off, junior engineer (or another agent, or another instance of Claude) picks up the plan and executes. That's the workflow most engineering teams have been waiting for.

What was theater

The "Claude can write your entire startup in 4 hours" demo. It can't. The demo was real but the codebase was a tutorial-grade CRUD app with no auth, no RBAC, no rate limiting, no observability, no error handling, and (critically) no business logic that wasn't representable as a single SQL query. Every one of those is the part where production AI coding still falls over. The demo was honest about being a tutorial, but the headline write-ups won't be.

The "$100B GDP" framing. Anthropic's CEO talked about Claude reaching 100% of global GDP within 21 months. That's not a claim about today; it's a claim about projection curves. It made for a viral clip. It tells you exactly nothing about whether you should build with Claude Code today versus Cursor versus a homegrown setup. Pick on the basis of the actual feature set you saw in the lab demos, not the keynote rhetoric.

The agentic IDE preview. Anthropic showed an early IDE-integrated experience that looked very polished. It's not shipping in 2026. The team confirmed that, off-stage, when pressed. The demo is real, the timeline is not.

Show the mechanism

Why background sub-agents matter, in concrete terms:

A typical agentic-coding session, before yesterday, ran into a wall around 60–80k tokens of context. Past that, your effective reasoning quality degraded — Claude started forgetting earlier decisions, repeating the same investigation paths, and (most subtly) losing track of which files it had already touched in the current session. The fix was usually a hard restart with a clean context, which lost continuity.

Background sub-agents change the math. Now the heavy investigation work — auditing 500 files, running an entire test suite and analyzing failures, reading a year of git log to understand a refactor history — happens in a separate context, and only the *summary* returns to your main session. That summary might be 200 tokens. Your main session stays in the 30–40k range, which is the comfortable zone for most reasoning tasks.

The mental model: **the sub-agent is a research assistant.** You don't read every paper your assistant reads. You read their two-paragraph summary and the citations they flagged.

The trap nobody is warning about

If you're building production agents that use Claude Code-style sub-agents, watch for **summary loss**.

When a sub-agent returns a 200-token summary of work it did across 50,000 tokens of context, that summary is necessarily lossy. Specifically: it loses *the thing you didn't ask about*. A sub-agent told to "find security antipatterns in this directory" will report security findings — but won't surface a critical performance issue it walked past, even though it spent two hours staring at the relevant code.

This is the same failure mode that plagues human research teams. The senior engineer who delegates a research task only learns what the junior engineer thought to surface. The unknown unknowns stay unknown. With AI sub-agents, the bandwidth gap between what they processed and what they reported is even wider, so the unknowns are even more numerous.

Two mitigations worth adopting:

Re-prompt the sub-agent for orthogonal axes. After a security audit, ask the same sub-agent (or a fresh one, with the same context window) to do a performance audit on the same code. The cost is one extra invocation; the catch rate is meaningfully better.

Keep the sub-agent's working file. Most sub-agent frameworks let you persist the agent's intermediate scratchpad, not just the final summary. Read the scratchpad when the summary feels suspiciously clean. The good stuff is often in the rejected branches of the investigation.

Who should care

- **Engineering teams shipping with agentic coding tools** (Claude Code, Cursor, Aider, Devin, etc.): the background sub-agent pattern is now generally available, and your team workflow should incorporate it within two weeks. The hand-off bottleneck just got smaller; use it.
- **Solo builders running Claude Code on side projects**: the plan-mode hand-off is the under-marketed feature most worth your time. Stop redoing the planning work every time you start a session. Save the plans, version them in git, re-feed them.
- **Tool builders working on agent SDKs**: the skill system is a forcing function. If your agent framework can't load and invoke a Claude-style skill manifest, build that compatibility now. The community library is going to have hundreds of skills within six months, and your framework's value drops significantly if it can't share that ecosystem.

The Code w/ Claude event was a quiet release event with one big feature (background sub-agents) buried under polish theater. That's actually a good signal. Frontier labs talk loudest about the things that aren't quite ready and quietest about the things that just shipped to production. The skills system, the durable plans, the background agents — those are all in production now. Build with them this week.

If you're putting agents into production and trying to figure out the multi-agent coordination patterns, that's the territory we covered in Agent Memory: The 5 Patterns That Ship in

Production. The decision tree for sub-agents — what to delegate, how to summarize back, when the parent agent should re-investigate — is the same problem regardless of whether you're using Claude Code or rolling your own.

— AI CODING

The best books for building AI agents in 2026

By **Flowi Editorial** · *May 9, 2026 · 6 min read*

An honest list of the books worth reading if you're shipping AI agents to production this year. What each covers, where each falls short, and the gap none of them fills.

The AI agent literature in 2026 is in an awkward place. The frontier moves quarterly. Most published books are 18-24 months out of date by the time they hit Kindle. The good content lives in scattered blog posts, conference talks, and a small set of recent books that have aged surprisingly well.

This is the honest list. Six books worth your reading time if you're shipping agents to production right now. What each is good for, where each is dated, and the gap none of them fills.

The criteria

A book qualifies for this list if it answers at least one of:

1. **What's the architecture pattern** for an agent that actually ships?

2. **What's the failure mode** for that pattern, and how to engineer around it?
3. **What's the right mental model** for the underlying transformer / agent / tool-use system?

Books that are general "AI for business" overviews don't qualify — they're outdated the day they print. Books that are pure research surveys don't qualify — they're useful but they don't help you ship.

The six books

1. **"Designing Machine Learning Systems"** — Chip Huyen, O'Reilly, 2022. Still the strongest single book on production ML systems generally. Agents are a subset.
2. **"Building LLM Applications for Production"** — Suhas Pai, Manning, 2024. The most current general overview; the production patterns chapter is the best part.
3. **"AI Engineering"** — Chip Huyen, O'Reilly, 2025. The follow-up to Designing ML Systems, written for the LLM era. Less generally-applicable than its predecessor but more relevant to agents specifically.
4. **"Building Multimodal AI Applications"** — Tarun Sharma, Packt, 2025. The only book that genuinely covers the multimodal patterns (vision-language-tool agents) honestly. Some chapters are thinner than others.
5. **"Hands-On Large Language Models"** — Jay Alammar & Maarten Grootendorst, O'Reilly, 2024. The best mental-model book — explains what's happening inside the transformer well enough that you stop being confused by edge cases.
6. **"Agent Memory: The 5 Patterns That Ship in Production"** — Flowi Editorial, 2026. Disclosure: we wrote this. The first book focused specifically on the memory layer that breaks at message four.

"Designing Machine Learning Systems" — Chip Huyen

The case for: Even three years out, the framework for thinking about production ML is unmatched. The chapters on data engineering, feature stores, and monitoring apply directly to agent systems with minimal translation.

The case against: Pre-LLM era. The model-training emphasis is overweighted relative to where agent work actually lives now.

Best for: Engineers transitioning from traditional ML into LLM/agent work. The mental models transfer well; this book is the bridge.

"Building LLM Applications for Production" — Suhas Pai

The case for: Solid coverage of RAG, tool-use, and basic agent patterns. The chapter on evaluation is unusually honest about how hard it is.

The case against: Tries to cover too much. The depth on any specific pattern is shallow. By the time you're past the basics, you've outgrown the book.

Best for: Engineers shipping their first production LLM application. Skip if you've already done two or three.

"AI Engineering" — Chip Huyen

The case for: The most current treatment of the production AI stack. Excellent treatment of agentic workflows, fine-tuning decisions, and the LLMOps tooling landscape.

The case against: The agent-specific patterns chapter is good but not deep. You'll need to supplement with focused books or papers for any specific pattern.

Best for: Mid-level engineers building real systems, who want a textbook-level treatment of the stack.

"Building Multimodal AI Applications" — Tarun Sharma

The case for: The vision-language sections are the best published treatment of the topic. Real code, real failure modes, honest about what's hard.

The case against: Chapters on speech and embodied agents are thinner. The book wants to be a complete multimodal reference but only really delivers on vision-language.

Best for: Engineers shipping agents that need to reason over images or screenshots. Skip the back half.

"Hands-On Large Language Models" — Jay Alammar & Maarten Grootendorst

The case for: The best book for *understanding what's actually happening*. Alammar's visualization approach (he wrote "The Illustrated Transformer") translates to book-length and clarifies a lot of confusion.

The case against: Mental-model book, not a shipping book. You'll close it understanding the transformer better but not knowing more about production patterns.

Best for: Engineers who keep getting confused by edge cases ("why did the model do that?") and want the deeper intuition.

"Agent Memory: The 5 Patterns That Ship in Production"

Disclosure: we wrote this. Take the recommendation with that in mind.

The case for: The only book focused entirely on the memory layer. The 5 patterns (Conversation Memory, Summary Memory, Vector Memory, Knowledge Graph Memory, Hybrid Memory) are the patterns you actually need; the failure modes named for production systems are the failure modes you'll hit. Short — 4,500 words, one weekend to read, code that runs.

The case against: Narrow scope by design. If you want a book that covers tools, planning, evaluation, *and* memory, this isn't it (yet — we're writing those).

Best for: Engineers whose agents work in demos but fail at message four. Specifically the *memory* failure mode. If your agent is failing at tool use or planning, this book won't help directly.

The gap none of these fills

No book in the category covers **how to evaluate AI agents in production before they fail**. Every book has an "evaluation" chapter that's mostly theoretical — generic benchmarks, golden datasets, the classic ML stuff. None of them treat evaluation as the *operational discipline* it actually is — pre-deployment behavioral tests, ongoing regression suites, what to do when your eval breaks before your agent does.

This is where the next major book in the space needs to be written. We're working on one (book №04 in the Flowi roadmap). Until then, the gap lives in scattered blog posts, mostly from the Anthropic and OpenAI engineering teams, plus a few Substack writers (Eugene Yan, Hamel Husain, Simon Willison).

How to pick

The honest decision tree by what you most care about:

- **Bridging from traditional ML** → Designing Machine Learning Systems
- **First-time LLM application** → Building LLM Applications for Production
- **Modern stack overview** → AI Engineering
- **Vision-language agents** → Building Multimodal AI Applications
- **Understanding the transformer** → Hands-On Large Language Models

- **Agent memory specifically** → Agent Memory: The 5 Patterns That Ship in Production

A recommendation: **don't read more than two books at once.** The category moves fast enough that depth matters more than breadth. Pick the one that addresses your current production bottleneck. Read it. Ship the thing. Pick the next one.

Who should care

- **Engineers shipping their first agent:** start with Building LLM Applications for Production. Move to AI Engineering when you're past the basics.
- **Engineers whose agent works in demo but fails in production:** the issue is almost always memory, tools, or evaluation. Memory is what our book covers.
- **Solo and small-team builders:** budget book money. $19-30 per book is rounding error for the time you save not figuring out what others have already figured out.

The AI agent book market is full of titles that don't survive the next quarter. The six above are the ones that have aged or will age — pick by your specific gap.

If your agent forgets the user at message four — which is the most consistent failure mode across every "AI agent demo" — that's the territory Agent Memory: The 5 Patterns That Ship in Production was built for. The decision tree, the code, and the failure modes nobody warns you about. $19, 4,500 words, one weekend, ships on Monday.

— IF THIS WAS USEFUL

The agent memory patterns that ship in production.

Most AI agents fail at message four because the agent forgets the user. Our book “Agent Memory: The 5 Patterns That Ship in Production” covers the decision tree, the code, and the failure modes nobody warns you about — in five chapters and 4,500 words of running code.

Read it — $19 →

The AI Builder’s Field Guide — a Flowi field guide.
Articles originally published on useflowi.app/blog.
Compiled for offline reading. Updated annually.